PRISM-EM for Non-Stationary Poisson GLMs with states from Spike Trains: Model, Objective, and Implementation. Ver3

Jan Balewski, NERSC

June 10, 2026

Abstract

This document describes the current `prism_EM_train3.py` implementation (script header: `prism_EM_train.py`) for fitting a non-stationary Poisson GLM with latent state mixing (PRISM-EM) [1]. Input spike trains are read from `spikesData/<dataName>.spikes.npz` (non-stationary output of `gen_nonStationarySpikes3.py`). The model uses a shared connectivity matrix across states, state-specific bias vectors, and time-varying simplex coefficients. Training is performed by block coordinate descent: a sequential projected-gradient E-step for coefficients followed by Viterbi decoding to one-hot state assignments, and an Adam-based M-step for model parameters with off-diagonal L1 shrinkage and spectral-radius projection. Spectral-radius enforcement, L1 pruning, and learning-rate decay are each activated after a configurable number of EM iterations. Distributed training (`torchrun`) shards the E-step over time bins and synchronizes A, B, and c across GPUs. We summarize the exact objective used in code, the decoding method, all current default hyperparameters, initialization modes (data-driven vs random) for A, B, and C, and the saved fit file layout.

Contents

1 EM Training for Non-Stationary Poisson GLM with states **2**
- 1.1 Introduction and Model Assumptions . . . 2
- 1.2 Objective Function . . . 2
- 1.3 Why Block Coordinate Descent (EM) over Joint Optimization . . . 2
- 1.4 Algorithm: Block Coordinate Descent (EM) . . . 3
  - 1.4.1 E-step: Coefficient Inference via Simplex PGD, then One-Hot Encoding . . . 3
  - 1.4.2 M-step: Dictionary Update via Adam . . . 4
- 1.5 State Decoding . . . 4

2 Default Hyperparameters and Runtime Settings **4**

3 Initialization of Model **5**
- 3.1 State-Coefficient Initialization ($C \equiv \{c_t\}$) . . . 5
- 3.2 Bias Initialization (B) . . . 6
- 3.3 Connectivity Initialization (A) . . . 6

4 Output File and Evaluation **6**

[1] PRISM: PRobabilistic Inference of Sparse Mixed-state dynamics; EM: Expectations Maximization

1 EM Training for Non-Stationary Poisson GLM with states

1.1 Introduction and Model Assumptions

We consider a population of N neurons observed over T discrete time bins of width Δt (seconds). The population activity is governed by M latent dynamical states that switch over time. The key structural assumption is that all states share a single connectivity matrix $A \in \mathbb{R}^{N\times N}$ but differ in baseline excitability through per-state bias vectors $B_m \in \mathbb{R}^N$, $m = 0, \ldots, M-1$ (0-based state indices in code).

At each time bin t, a coefficient vector $c_t \in \mathbb{R}^M$, constrained to the probability simplex ($c_{mt} \geq 0$, $\sum_m c_{mt} = 1$), selects the effective bias as a convex combination:

$$B_t = \sum_{m=1}^{M} c_{mt}\, B_m = c_t^\top \mathbf{B}, \tag{1}$$

where $\mathbf{B} \in \mathbb{R}^{M\times N}$ stacks the M bias vectors row-wise.

Spike counts follow

$$\eta_t = A\, Y_{t-1} + B_t, \qquad \tilde{\eta}_t = \min(\eta_t, \eta_{\text{clip}}), \qquad \mu_t = \exp(\tilde{\eta}_t)\, \Delta t, \qquad Y_t \sim \text{Poisson}(\mu_t). \tag{2}$$

Notation summary. N: number of neurons; M: number of latent states; T: number of time bins; Δt: time-bin width (from metadata); η_{clip}: clipping threshold (from metadata); $Y_t \in \mathbb{N}_0^N$: observed spikes; $\mu_t \in \mathbb{R}_+^N$: expected spike count per bin.

1.2 Objective Function

The training objective uses uniformly weighted (unweighted) Poisson NLL in both the E-step and M-step:

$$\mathcal{J} = \underbrace{\sum_{t=2}^{T}\sum_{i=1}^{N} [\mu_{t,i} - Y_{t,i}\,(\tilde{\eta}_{t,i} + \log \Delta t)]}_{\text{Poisson NLL}} + \underbrace{\lambda_2 \sum_{t=2}^{T} \|\vec{c}_t - \vec{c}_{t-1}\|^2}_{\text{temporal smoothness (E-step only)}} + \underbrace{\lambda_3\, \overline{|A_{\text{off}}|}}_{\text{L1 sparsity on off-diagonal (M-step only)}}, \tag{3}$$

with simplex constraints $c_t \in \Delta^{M-1}$ and spectral constraint $\rho(A) \leq \rho_{\max}$, where the off-diagonal L1 term is

$$\overline{|A_{\text{off}}|} = \frac{1}{N(N-1)} \sum_{\substack{i,j=1 \\ i\neq j}}^{N} |A_{ij}|. \tag{4}$$

In the λ_2 term, the sum over states m is already implicit in the L_2 norm.

1.3 Why Block Coordinate Descent (EM) over Joint Optimization

In principle one could jointly optimize A, $\mathbf{B}$, and $c_{1:T}$ by simultaneous gradient descent on the full objective $\mathcal{J}$. In practice, the EM-style alternation is preferable for three structural reasons.

Breaking the bilinear coupling. The coefficients c_t and the bias matrix $\mathbf{B}$ enter the log-rate linearly through the product $c_t^\top \mathbf{B}$, which then passes through an exponential. When both are free to move, the landscape contains saddle points and flat valleys: a large c_{tm} paired with a small B_m produces the same likelihood as the reverse. Fixing one side at each step eliminates this ambiguity. The Viterbi hard-assignment variant sharpens the separation further: with one-hot $c_t = e_{\hat{S}_t}$, each time bin selects exactly one bias row, so the M-step reduces to M independent Poisson GLM regressions and the bilinear degeneracy vanishes entirely.

Exploiting temporal structure in the E-step. The coefficient vectors $c_{1:T}$ number $O(TM)$ and vastly outnumber the $O(N^2+MN)$ model parameters. Joint optimization would need to balance learning rates across these disparate groups while enforcing a simplex constraint at every time bin. By contrast, the E-step decomposes into a single forward sweep of cheap M-dimensional simplex-projected gradient updates, naturally incorporating the temporal smoothness penalty $\lambda_2\|c_t - c_{t-1}\|^2$ through the sequential dependence on the previous accepted c_{t-1}.

Avoiding the log-sum-exp mismatch. Under soft (probabilistic) state assignments the M-step must fit parameters to the mixture rate $\sum_m c_{tm}\exp(\eta_{tm})$, whereas the model parameterizes the log-rate as $\sum_m c_{tm}\,\eta_{tm}$ inside a single exponential. These two quantities differ whenever the assignment is not one-hot, creating a systematic bias in the gradient signal for $\mathbf{B}$. The hard-EM strategy—Viterbi decode followed by one-hot encoding—eliminates this mismatch by construction, at the cost of losing the monotonic likelihood improvement guarantee of soft EM, a trade-off that is empirically favorable.

1.4 Algorithm: Block Coordinate Descent (EM)

The non-convex problem is solved by alternating:

- E-step: update $c_{1:T}$ with $A, \mathbf{B}$ fixed, then Viterbi-decode to one-hot state assignments.
- M-step: update $A, \mathbf{B}$ with one-hot $c_{1:T}$ fixed.

for K_{EM} outer iterations.

1.4.1 E-step: Coefficient Inference via Simplex PGD, then One-Hot Encoding

Holding $(A, \mathbf{B})$ fixed, coefficients are inferred by one forward sweep from $t = 2$ to T. At each time bin:

1. Build $Z_t[m, :] = A\,Y_{t-1} + B_m \in \mathbb{R}^{M\times N}$.
2. Run `pgd_iter` projected-gradient iterations:

$$\eta_t = c_t^\top Z_t, \quad \tilde{\eta}_t = \min(\eta_t, \eta_{\text{clip}}), \quad \mu_t = \exp(\tilde{\eta}_t)\Delta t,$$

$$g_t = Z_t\,(\mu_t - Y_t) + 2\lambda_2(c_t - c_{t-1}), \qquad c_t \leftarrow \Pi_{\Delta^{M-1}}(c_t - \alpha_E g_t).$$

3. Accept c_t, continue to next time bin.

Projection uses the sort-based simplex projection (Duchi et al., $O(M\log M)$). Bin $t = 0$ is not updated; the sweep starts at $t = 1$ (first lag pair (Y_0, Y_1)), matching the objective sum over $t = 2, \ldots, T$ in 1-based notation. In the distributed setting, each rank updates a disjoint time shard of $c_{1:T}$; shard results are averaged where multiple ranks overlap at shard boundaries, then broadcast so all ranks share the same c.

After the forward sweep completes, the soft coefficients $c_{1:T}$ are decoded into a hard state sequence via Viterbi (Section 1.5). The M-step then receives one-hot encoded state assignments rather than the soft simplex probabilities:

$$\hat{S}_{1:T} = \text{Viterbi}(c_{1:T}), \qquad c_t^{(\text{M-step})} = e_{\hat{S}_t}, \tag{5}$$

where e_m is the m-th standard basis vector. This ensures the M-step fits each state's parameters from cleanly separated data rather than from soft mixtures.

1.4.2 M-step: Dictionary Update via Adam

Holding c_t fixed (as one-hot vectors), update $A, \mathbf{B}$ for K_M epochs using Adam over shuffled mini-batches of (Y_{t-1}, Y_t, c_t).

Per mini-batch:

$$\ell_{\mathrm{M}} = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \sum_{i=1}^{N} \left[-Y_{t,i} \log(\mu_{t,i} + \varepsilon) + \mu_{t,i} \right] + \lambda_3 \, \overline{|A_{\mathrm{off}}|}, \tag{6}$$

with $\varepsilon = 10^{-8}$.

Then, subject to delay scheduling (operations are activated only after a configurable number of EM iterations have passed):

1. Off-diagonal soft-thresholding (activated after K_{prune} EM iterations):

$$A_{ij} \leftarrow \mathrm{sign}(A_{ij}) \left(|A_{ij}| - \alpha_M \lambda_3 \right)_+, \quad i \neq j.$$

2. Periodic spectral-radius projection (activated after K_ρ EM iterations, applied every R batches). On each application, in the multi-GPU setting, A and B are first averaged across ranks, then the spectral constraint is enforced, and the result is broadcast:

$$A \leftarrow A \cdot \min\left(1, \frac{\rho_{\max}}{\rho(A)}\right).$$

Learning rate follows linear decay from $\alpha_M^{(0)}$ to $\alpha_M^{(0)} f_{\mathrm{end}}$. The decay is activated only after K_{lr} EM iterations have passed; the total number of decay steps spans the remaining M-step epochs.

1.5 State Decoding

Viterbi decoding is applied both within the EM loop (after each E-step, to produce one-hot assignments for the M-step) and after the final EM iteration (to produce the output state sequence). The decoder uses a homogeneous transition prior with $\tau_{\mathrm{dwell}} =$ `decode_dwell_sec`:

$$p_{\mathrm{stay}} = \exp\left(-\frac{\Delta t}{\tau_{\mathrm{dwell}}}\right), \qquad p_{\mathrm{switch}} = \frac{1 - p_{\mathrm{stay}}}{M - 1}. \tag{7}$$

The decoded sequence is

$$\hat{S}_{1:T} = \arg\max_{\{s_t\}} \sum_t \left[\log c_{t,s_t} + \log p(s_t \mid s_{t-1}) \right]. \tag{8}$$

Per-bin confidence is

$$\mathrm{CL}_t = 1 - \max_m c_{t,m}.$$

2 Default Hyperparameters and Runtime Settings

Required input. `--num_states` (`-M`) is required (no default). `--dataName` and `--basePath` locate input spikes under `<basePath>/spikesData/`; fits are written to `<basePath>/prismFit/<fitName>.prismEM.npz`. Wrapper script `fitPrismEM.sh` launches distributed training via `torchrun` (default 4 GPUs) and sets `--time_range_sec 0 80` unless overridden.

Dataset-provided values. Δt (`time_step_sec`) and η_{clip} (`poisson_eta_clip`) are loaded from spike-file metadata.

Group	Parameter (CLI)	Default
EM structure	--num_em_iters	20
	--m_epochs	100
E-step	--pgd_iter	5
	--lr_estep	0.03
	--lambda2	2.0
M-step	--lr_mstep	3×10^{-3}
	--lr_end_factor	0.1
	--lambda3	0.02
	--rho_max	0.95
	--prescale_m_step_4_ArhoMax	50 (batches)
	--delay_em_iter_4_ArhoMax	3
	--delay_em_iter_4_lrDecay	1
	--delay_em_iter_4_Aprune	1
	--batch_size	2048
	--minW	0.01 (reporting only)
Data window	--time_range_sec	[0.0, 60.0] s
Decoding	--decode_dwell_sec	0.3 s
Init	--init_A	data (or rand)
	--init_B	data (or rand)
	--init_states	data (or rand)

Table 1: Current defaults in prism_EM_train3.py.

3 Initialization of Model

Initialization is controlled by:

$$\texttt{--init\_states}, \quad \texttt{--init\_B}, \quad \texttt{--init\_A} \in \{\texttt{data}, \texttt{rand}\},$$

with default data for all three.

3.1 State-Coefficient Initialization ($C \equiv \{c_t\}$)

random mode:

$$c_t = \frac{1}{M}\mathbf{1}, \quad S_t = -1 \ \ \forall t.$$

So coefficients start uniform over states.

data mode:

1. Aggregate spikes into coarse blocks of length $\lceil \tau_{\text{dwell}}/\Delta t \rceil$ bins, where $\tau_{\text{dwell}} =$ decode_dwell_sec (default 0.3 s).
2. For each block, compute a scalar mean population firing rate.
3. Find $M-1$ thresholds minimizing within-cluster variance (dynamic-programming segmentation on unique rate values).
4. Assign each block to a state by thresholds.

5. Convert block states to soft probabilities: dominant state gets 0.8, remainder 0.2 split across others, with transition smoothing (2/3 vs 1/3 at boundaries).

6. Map block-level c and S back to original bin resolution.

3.2 Bias Initialization (B)

random mode: No external seed is applied; model keeps random Gaussian initialization from `torch.randn(...)*0`

data mode: For each neuron n:

$$r_n = \frac{1}{T}\sum_t Y_{t,n} \,/\, \Delta t, \qquad b_n = \log(\max(r_n, 10^{-6})).$$

The same b-vector is assigned to every state row of B.

3.3 Connectivity Initialization (A)

random mode: No external seed is applied; model keeps random Gaussian initialization from `torch.randn(...)*0`

data mode (covariance/OLS): Using first $T_{\max} = 50{,}000$ bins (or fewer if shorter):

$$A_{\text{ols}} = \left(\sum_t Y_t Y_{t-1}^\top\right)\left(\sum_t Y_{t-1} Y_{t-1}^\top\right)^\dagger.$$

Then all elements of A_{ols} are scaled by a factor of 3:

$$A \leftarrow 3\, A_{\text{ols}}.$$

No spectral-radius rescaling is applied at initialization; the spectral constraint is enforced later during training via the delayed projection mechanism.

Diagnostics (condition number, spectral radius, one-step R^2, Frobenius norm, bins used) are computed on the pre-scaled OLS estimate and saved in metadata (`init_A` sub-dictionary of the output metadata).

4 Output File and Evaluation

After training, rank 0 writes `<basePath>/prismFit/<fitName>.prismEM.npz`. If `--fitName` is omitted, the name is `<dataName>-EM-<6hex>`. Key arrays (shapes for T time bins, N neurons, M states):

- `A_init`, `A_hat`: (N, N);
- `B_init`: $(N,)$ — first row of the initialized B matrix (common across states in `data` mode);
- `B_hat`: (M, N) — full per-state bias matrix;
- `c_init`, `c_hat`: (T, M);
- `S_init`, `S_hat`: $(T,)$ integer state labels (`S_init` is -1 in `rand` init mode);
- `S_hat_CL`: $(T,)$ per-bin confidence $\text{CL}_t = 1 - \max_m c_{t,m}$;
- `e_nll_em`, `m_loss_epoch`, `m_nll_epoch`, `m_l1_epoch`, `rho_epoch`, `nz_edges_epoch`, `learning_rates`: training history.

Metadata records all CLI hyperparameters, time-window bins, initialization dictionaries, and `mstep_state_mode=vite`
Suggested evaluation:

```
./prism_EM_eval3.py --basePath $basePath --dataName <fitName> -p a e f g
```